Experiment 1.9

Mapped Course Outcomes-CO3

CO3: Identify the connection establishment techniques and features
AIM:- Using Socket Programming implement the connectionless oriented service using standard ports in any programming language (C,C++, JAVA,Python etc).

Apparatus Required:-Pycharm

The socket() Function

The first step is to call the socket function, specifying the type of communication protocol (TCP based on IPv4, TCP based on IPv6, UDP).

We plan to learn the following:

- What is a socket?
- The client-server model
- Byte order
- UDP socket API
- Concurrent server design
- Example of echo client and iterative server
- Example of echo client and concurrent server.

Prerequisites:-

1. A web browser with access to **C,C++, JAVA,Python etc**

<u>The function is defined as follows:</u>

```
#include <sys/socket.h>

int socket (int family, int

type, int protocol);
```

where family specifies the protocol family (AF_INET for the IPv4 protocols), type is a constant described the type of socket (SOCK_STREAM for stream sockets and SOCK_DGRAM for datagram sockets.

The function returns a non-negative integer number, similar to a file descriptor, that we define socket descriptor or -1 on error.

The connect() Function

The connect() function is used by a TCP client to establish a connection with a TCP server/

The function is defined as follows:

#include <sys/socket.h> int connect (int sockfd, const

struct sockaddr *servaddr, socklen_t addrlen); where

sockfd is the socket descriptor returned by the socket

function.

The function returns 0 if the it succeeds in establishing a connection (i.e., successful TCP threeway handshake, -1 otherwise.

The client does not have to call bind() in Section before calling this function: the kernel will choose both an ephemeral port and the source IP if necessary.

The send() Function

Since a socket endpoint is represented as a file descriptor, we can use read and write to communicate with a socket as long as it is connected. However, if we want to specify options we need another set of functions.

For example, send() is similar to write() but allows to specify some options. send() is defined as follows:

#include <sys/socket.h> ssize_t send(int

sockfd, const void *buf, size_t nbytes, int

flags);

where buf and nbytes have the same meaning as they have with write. The additional argument flags is used to specify how we want the data to be transmitted. We will not consider the possible options in this course. We will assume it equal to 0.

The function returns the number of bytes if it succeeds, -1 on error.

The receive() Function

The recv() function is similar to read(), but allows to specify some options to control how the data are received. We will not consider the possible options in this course. We will assume it is equal to 0.

receive is defined as follows: #include

<sys/socket.h> ssize_t recv(int sockfd,

void *buf, size_t nbytes, int flags);

The function returns the length of the message in bytes, 0 if no messages are available and peer had done an orderly shutdown, or -1 on error.

The close() Function

The normal close() function is used to close a socket and terminate a TCP socket. It returns 0 if it succeeds, -1 on error. It is defined as follows:

#incl

ude

<unis

td.h>

int

close(

int

sockf

d);

Coding Implementation: Exchange of hello message between server and client is shown to demonstrate the connection-less model.

// Server side implementation of UDP client-server model

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <sys/types.h>

#include <sys/socket.h>

#include <arpa/inet.h>

#include <netinet/in.h>

#define PORT 8080

#define MAXLINE 1024

/

/

D

r

i

v

e

r

c

o

d

e

i

n

t

```
m
a
i
n
(
)

{

i
n
t
s
o
c
k
f
d
;

c
h
a
r
```

b

u

f

f

e

r

[

M

A

X

L

I

N

E

]

;

c

h

a

r

*

h

e

l

l

o

=

"

H

e

l

l

o

f

r

o

m

s

e

r

v

e

r

"

;

s

t

r

u

c

t

s

o

c

k

a

d

d

r

_

i

n

s

e

r

v

a

d

d

r

,

c

l

i

a

d

d

r

;

```
        // Creating socket file descriptor
if ( (sockfd = socket(AF_INET, SOCK_DGRAM,
0)) < 0 ) {
                perror("socket creation failed");
exit(EXIT_FAILURE);
        }
        memset(&servaddr, 0, sizeof(servaddr));
memset(&cliaddr, 0, sizeof(cliaddr));
        // Filling server information
servaddr.sin_family = AF_INET; //
IPv4            servaddr.sin_addr.s_addr
```

```
= INADDR_ANY;
servaddr.sin_port = htons(PORT);

        // Bind the socket with the server
address         if ( bind(sockfd,
(conststructsockaddr *)&servaddr,
                        sizeof(servaddr)) < 0 )
        {
perror("bind failed");
exit(EXIT_FAILURE);
        }

i
n
t
l
e
n
,

n
;

        n = recvfrom(sockfd, (char *)buffer, MAXLINE,
                                MSG_WAITALL, ( structsockaddr *)
&cliaddr,
```

```
&len);                buffer[n] = '\0';
printf("Client : %s\n", buffer);
sendto(sockfd, (const char *)hello,
strlen(hello),
                        MSG_CONFIRM, (conststructsockaddr *) &cliaddr,
                                len);
            printf("Hello message sent.\n");

r
e
t
u
r
n

0
;

}
```

// Client side implementation of UDP client-server model

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
```

#include <sys/types.h>

#include <sys/socket.h>

#include <arpa/inet.h>

#include <netinet/in.h>

#define PORT 8080

#define MAXLINE 1024

/

/

D

r

i

v

e

r

c

o

d

e

i

n

t

m

a

i

n

(

)

{

intsockfd;

char buffer[MAXLINE];

char *hello = "Hello

from client";

structsockaddr_in

servaddr;

// Creating socket file descriptor

if ((sockfd = socket(AF_INET, SOCK_DGRAM,

0)) < 0) {

perror("socket creation failed");

exit(EXIT_FAILURE);

}

```
        memset(&servaddr, 0, sizeof(servaddr));

        // Filling server information
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(PORT);
servaddr.sin_addr.s_addr =
INADDR_ANY;                     int
n, len;

        sendto(sockfd, (const char *)hello, strlen(hello),
                MSG_CONFIRM, (conststructsockaddr *) &servaddr,

sizeof(servaddr));          printf("Hello
message sent.\n");

        n = recvfrom(sockfd, (char *)buffer, MAXLINE,
                                        MSG_WAITALL, (structsockaddr *)
&servaddr,

&len);          buffer[n] =
'\0';          printf("Server
: %s\n", buffer);


close(sockf
```

```
d);

return 0;

}
```

Further Reading-

1. https://realpython.com/python-sockets/

2.https://docs.python.org/3/howto/sockets.html